



EXTENSIBILITY FRAMEWORK

Abstract

Thongchai, Thanatporn (Bee)
Thanatporn.Thongchai@proalpha.com

Extensibility Framework

Document: [Instance Listener: UI - Documentation - Confluence](#)

Instance Listener คือ **Classes Repository** ที่สามารถกำหนดอ็อบเจกต์/คลาสให้กับโปรแกรม

(instances) ที่มีอยู่แล้วได้ ซึ่งเรียกว่า **Instance Listener**

คลาสเหล่านี้จะถูกสร้างอินสแตนซ์ (instantiate) ขึ้นภายหลังเมื่อโปรแกรมที่เกี่ยวข้องถูกเริ่มทำงาน และจะคอยรับฟัง (listen) เหตุการณ์ต่าง ๆ ที่เกิดขึ้นระหว่างการทำงานของโปรแกรม

ด้วยเหตุนี้ **Instance Listener** จึงสามารถใช้ในการแทรกโค้ดเพิ่มเติมเข้าไปในกระบวนการทำงานของโปรแกรมได้โดยไม่ต้องแก้ไขซอร์สโค้ดเดิม และสามารถใช้แทน **User Exit** ได้

ประเภทของเหตุการณ์ (Event Types)

เพื่อให้อ็อบเจกต์ประเภท **Instance Listener** สามารถตอบสนองต่อเหตุการณ์ (Events) ต่าง ๆ ได้ จำเป็นต้อง **implement Interface** ที่สอดคล้องกับประเภทของเหตุการณ์นั้น ๆ

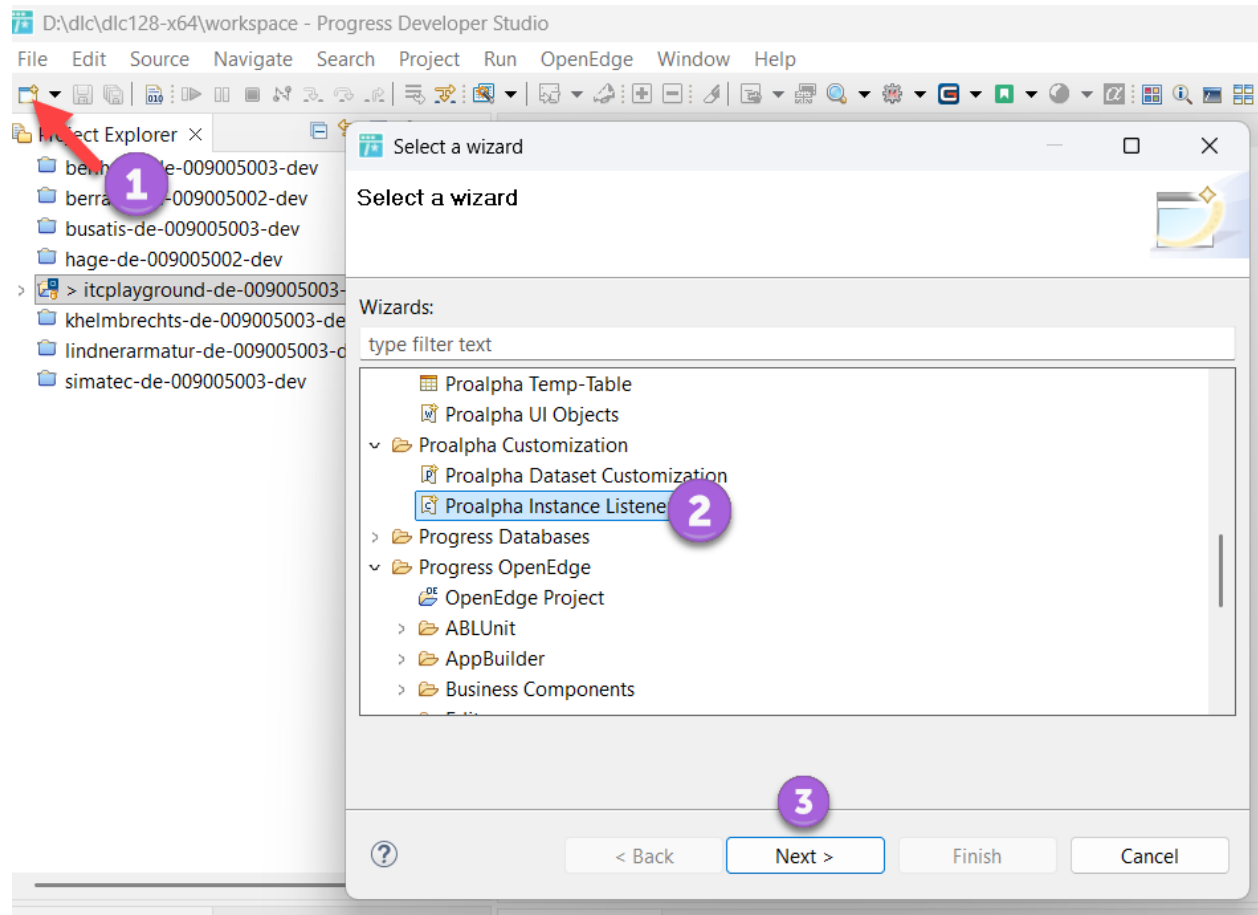
เมื่อคุณ **implement Interface** ที่เกี่ยวข้องแล้ว จะต้อง **implement** เมธอด **subscribe** ที่สอดคล้องกันด้วย ซึ่งเมธอดนี้ใช้สำหรับลงทะเบียน (subscribe) ให้อ็อบเจกต์นั้นทำหน้าที่เป็น **Listener** สำหรับเหตุการณ์ที่ต้องการสามารถอ่านรายละเอียดเพิ่มเติมได้ที่ **Document** ด้านบน

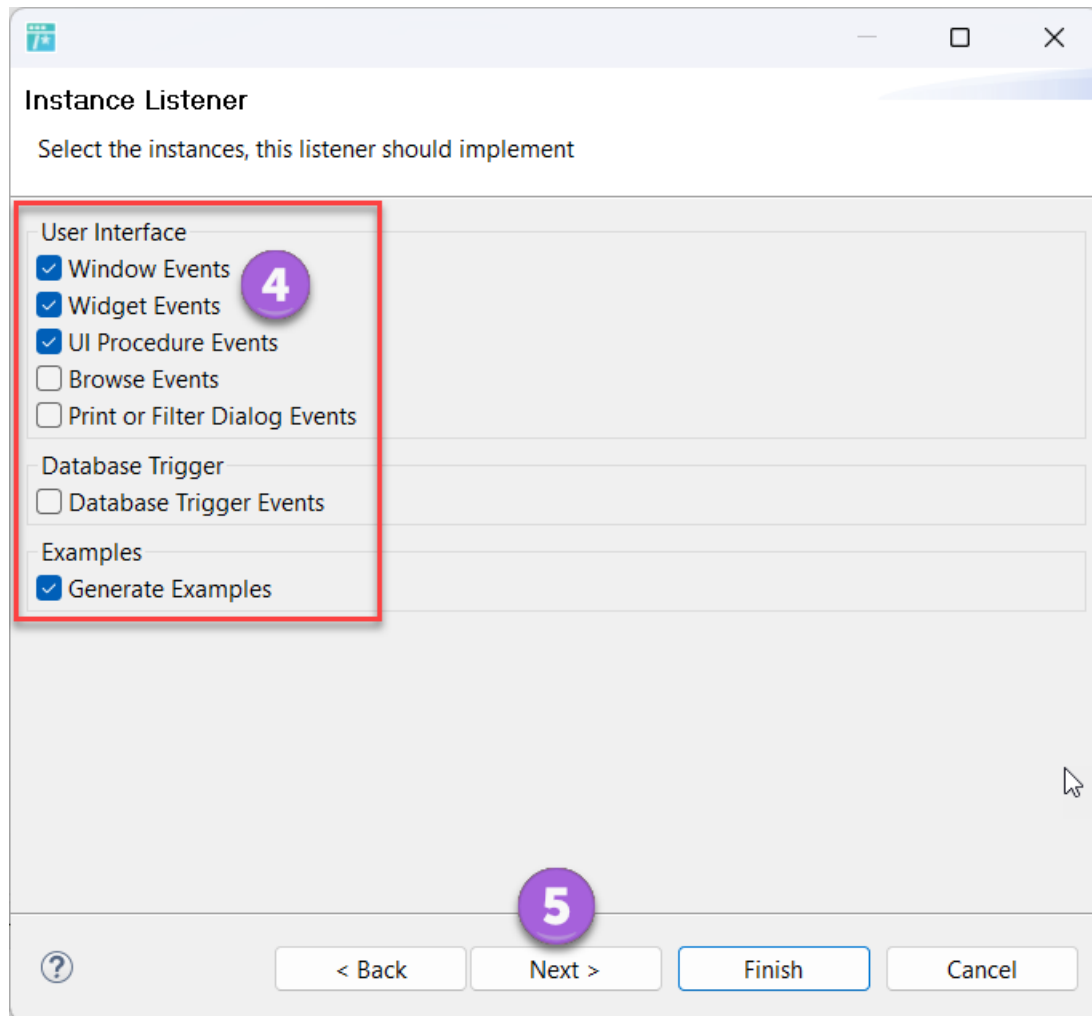
Creation Instance Listener**Naming convention: the class must end in "Lst"****(e.g XSC_s_wart00_w_Lst.cls)****X = level of customize คือ x เสมอ****S = Little of module (e.g s = stamm, v=vert)****C= Class****s_wart00_w = listener of UI s_wart00_w****Lst = class Listener**

Key message convert User-exit to Listener:

old	new
VORLAUF	PreProcessing
NARLAUF	PostProcessing

Create Class Listener





(4) : เลือก User interface ที่เราต้องการใช้งาน

เช่น สำหรับการเข้าถึง Toolbar ให้เลือกใช้ Windows

การเข้าถึง widget (field, button) ให้เลือกใช้ widget และ Procedure ให้เลือก UI Procedures

แต่ทั้งนี้ ทุก interface สามารถใช้ร่วมกันได้ และสามารถเข้าถึงกันได้หมด

Ex:

1. Move frame

1.1 Events Type: UI Procedure Events

1.2 Methods implemented

```
method public void subscribeUIProcedureEvents
( poUIProcedure as DMCUIProcedureSvo ):
/* Description -----*/
/*                               */
/* Subscription method for ui procedure events.                */
/*                               */
/* Parameters -----*/
/*                               */
/* poUIProcedure      Ui procedure instance.                    */
/*                               */
/*-----*/

poUIProcedure:prpoEvents.subscribeOnInitialize-PreProcessing(this-object, 'InitVorlauf':U).
poUIProcedure:prpoEvents.subscribeOnInitialize-PostProcessing(this-object, 'InitNachlauf':U).

end method.
```

1.3 Implemente code

```
method public void InitNachlauf
( poUIProcedure as DMCUIProcedureSvo ):
/* Description -----*/
/*                               */
/*                               */
/*                               */
/* Parameters -----*/
/*                               */
/* poUIProcedure      UI Procedure instance that triggered the event. */
/*                               */
/*-----*/

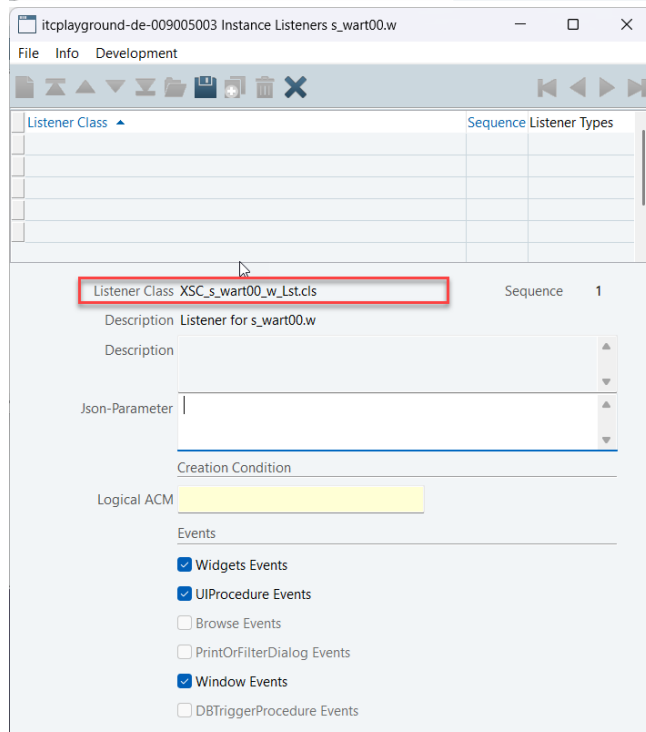
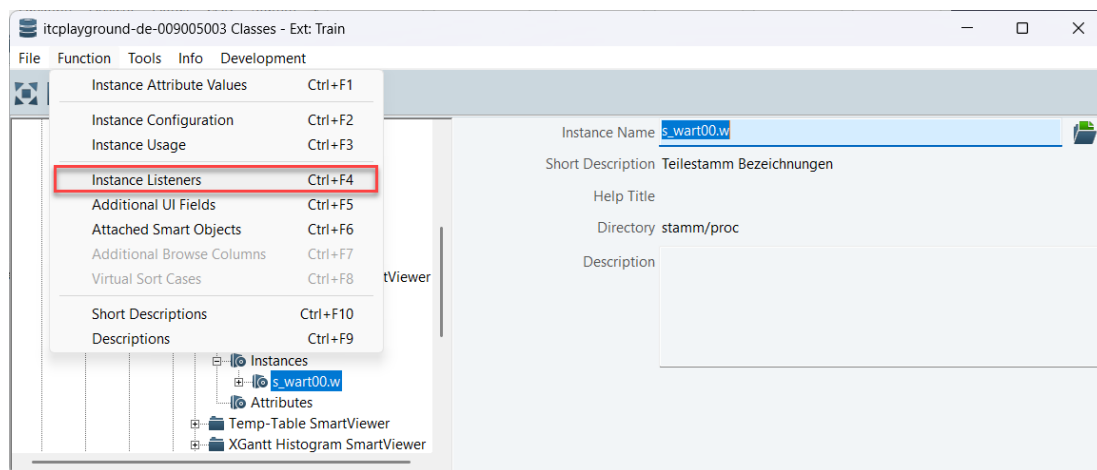
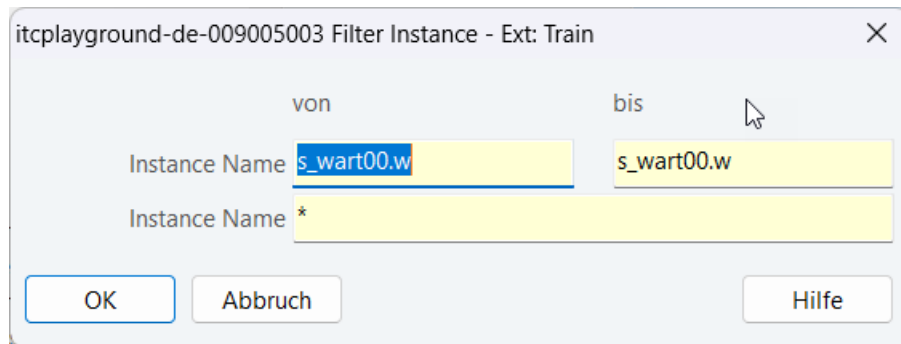
message "init nachlauf".

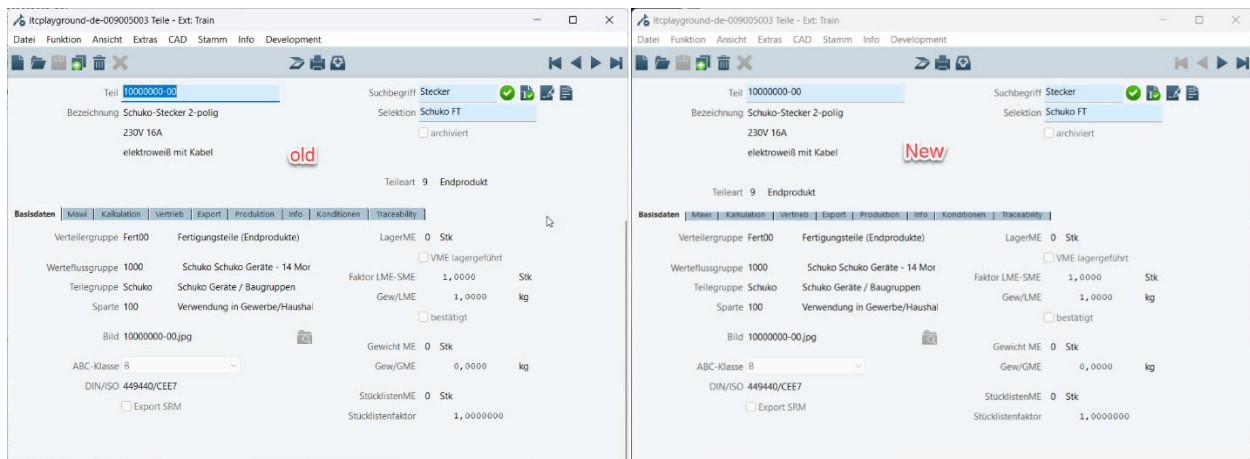
adm.method.cls.DMCUISvc.moveWidgetByPosRef
(poUIProcedure:oGetWidget('S_Artikel.ArtikelArt':U):prphWidget,
poUIProcedure:oGetWidget('S_ArtikelSpr.Bezeichnung[4]':U):prphWidget,
0,
1).|

end method.
```

หลังจากนั้น ให้มัน add listener ให้กับ program

ไปที่ repository class → program ที่ต้องการ





2. Adjust size frame

```
method public void InitVorlauf
( poUIProcedure as DMCUIProcedureSvo ):
/* Description ----- */
/*
/*
/*
/*
/* Parameters ----- */
/*
/* poUIProcedure      UI Procedure instance that triggered the event.
/*
/*----- */
message "init vorlauf".

poUIProcedure:setHeight (poUIProcedure:prphFrame:width-chars + 0.5) .

end method.
```

3. Create Widget

4. Tool bar

Example : ให้ลองเปลี่ยน User-exit จากโฟลท์ก่อนหน้ามาใช้ Listener